



Chương 5

ĐỒNG BỘ HÓA TIẾN TRÌNH

NỘI DUNG

- **Cơ sở**
- **Bài toán miền găng**
- **Giải pháp Peterson**
- **Khóa loại trừ (Mutex locks)**
- **Semaphore**
- **Các bài toán đồng bộ hóa kinh điển**
- **Monitor**
- **Đồng bộ hóa trên một số hệ điều hành**
- **Phương pháp tiếp cận thay thế**

MỤC TIÊU

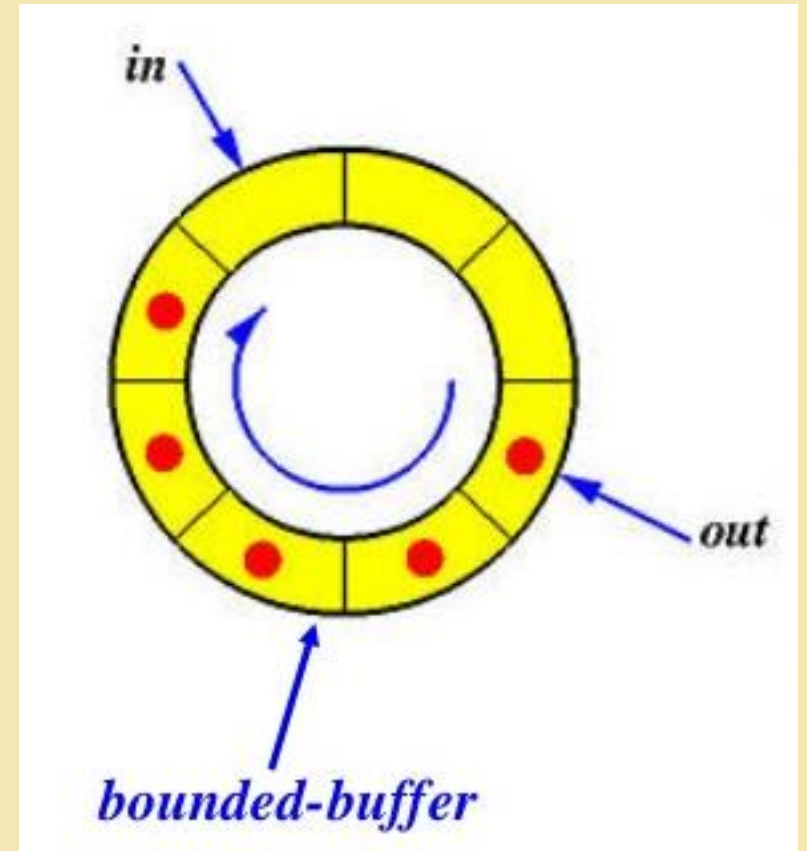
- ✓ Trình bày các khái niệm về đồng bộ hóa tiến trình
- ✓ Giới thiệu về bài toán miền găng, các giải pháp của nó có thể được sử dụng để đảm bảo tính nhất quán của dữ liệu chia sẻ
- ✓ Trình bày giải pháp của bài toán miền găng
- ✓ Xem xét một số bài toán đồng bộ hóa kinh điển
- ✓ Khám phá nhiều công cụ được sử dụng để giải quyết bài toán đồng bộ hóa tiến trình

Cơ sở

- Các tiến trình có thể thực thi đồng thời
 - Có thể bị gián đoạn bất cứ lúc nào, hoàn thành 1 phần việc thực thi.
- Truy cập đồng thời đến dữ liệu chia sẻ có thể dẫn đến dữ liệu không nhất quán (mâu thuẫn)
- Duy trì tính nhất quán dữ liệu đòi hỏi các cơ chế đảm bảo việc thực thi của các tiến trình hợp tác có thứ tự

Bài toán Producer-Consumer

- Tiến trình Producer sản xuất thông tin được tiêu thụ bởi tiến trình Consumer
- Giả sử ta có bộ đệm vòng (mảng vòng) với n ô nhớ.
- Con trỏ **IN** trỏ đến ô trống kế tiếp trong vùng đệm, con trỏ **OUT** trỏ đến ô đầy đầu tiên trong vùng đệm.
- Tiến trình Producer có nhiệm vụ thêm thông tin vào vùng đệm.
- Tiến trình Consumer có nhiệm vụ lấy thông tin từ vùng đệm.



Bài toán Producer-Consumer

```
#define BUFFER_SIZE 8
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```


Bài toán Producer-Consumer

```
item next_produced;
while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

Bài toán Producer-Consumer

```
item next_consumed;
while (true) {
    while (in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next consumed */
}
```


Minh họa vấn đề

- Giải thuật của bài toán cho phép tối đa `BUFFER_SIZE - 1` thông tin trong ô nhớ tại cùng một thời điểm.
- Giả sử chúng ta muốn cung cấp một giải pháp cho bài toán **consumer-producer** là lấp đầy tất cả các ô nhớ vùng đệm. Chúng ta có thể làm như vậy bằng cách dùng một biến **counter** kiểu số nguyên để theo dõi số lượng các ô vùng đệm. Ban đầu, **counter** được thiết lập là 0. Nó được gia tăng bởi **producer** sau khi thêm thông tin vào một ô vùng đệm và giảm đi bởi **consumer** sau khi lấy thông tin từ một ô vùng đệm.

Producer

```
while (true) {  
    /* produce an item in next produced */  
  
    while (counter == BUFFER_SIZE) ;  
        /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

Consumer

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed  
*/  
}
```

Tranh đoạt điều khiển (race condition)

- Counter++ có thể được thực hiện như sau

```
register1 = counter  
register1 = register1 + 1  
counter = register1
```

- Counter-- có thể được thực hiện như sau

```
register2 = counter  
register2 = register2 - 1  
counter = register2
```

- Xem xét việc thực hiện đan xen với "counter = 5" ban đầu

T0: producer thực thi	register1 = counter	{register1 = 5}
T1: producer thực thi	register1 = register1 + 1	{register1 = 6}
T2: consumer thực thi	register2 = counter	{register2 = 5}
T3: consumer thực thi	register2 = register2 - 1	{register2 = 4}
T4: producer thực thi	counter = register1	{counter = 6}
T5: consumer thực thi	counter = register2	{counter = 4}

Tranh đoạt điều khiển

- Tình huống xuất hiện khi nhiều tiến trình cùng thao tác trên dữ liệu chung và kết quả các thao tác đó phụ thuộc vào thứ tự thực hiện của các tiến trình được gọi là tranh đoạt điều khiển (race condition)
- Để tránh các tình huống tranh đoạt điều khiển, các tiến trình cần được đồng bộ theo một phương thức nào đó $\Rightarrow$ Vấn đề nghiên cứu: Đồng bộ hóa các tiến trình

Bài toán miền găng (Critical Section)

- Xem xét hệ thống với n tiến trình $\{p_0, p_1, \dots, p_{n-1}\}$
- Mỗi tiến trình có một đoạn mã là CS
 - Tiến trình có thể thay đổi các biến chung, cập nhật bảng, ghi tập tin,...
 - Khi một tiến trình trong CS, không có tiến trình nào khác có thể trong CS của nó
- Bài toán miền găng là yêu cầu thiết kế giao thức để giải quyết điều này
- Mỗi tiến trình phải yêu cầu quyền để vào miền găng của nó thông qua **entry section**. Theo sau miền găng có thể là **exit section**, tiếp theo là **remainder section**

Miền găng

- Cấu trúc tổng quát của tiến trình P_i

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

Giải pháp cho bài toán miền găng

- Độc quyền truy xuất (Mutual Exclusion) – nếu tiến trình P_i đang thực thi trong miền găng của nó thì không có tiến trình nào khác thực thi trong miền găng của chúng
- Tiến độ (Progress) – nếu không có tiến trình nào thực thi trong miền găng của nó và tồn tại một số tiến trình muốn vào miền găng của chúng thì việc lựa chọn tiến trình tiếp theo vào miền găng không thể bị trì hoãn vô hạn
- Giới hạn chờ (Bounded waiting) – tồn tại một ràng buộc hoặc giới hạn số lần vào CS của các tiến trình khác sau khi một tiến trình thực hiện yêu cầu vào CS và trước khi yêu cầu đó được chấp nhận
 - Giả sử rằng mỗi tiến trình thực hiện với một tốc độ khác 0
 - Không có giả thiết nào đặt ra cho sự liên hệ về tốc độ của các tiến trình

Quản lý miền găng trong hệ điều hành

- Hai cách tiếp cận
 - Không độc quyền (preemptive) – cho phép tiến trình được ưu tiên khi chạy trong chế độ nhân
 - Độc quyền (nonpreemptive) – không cho phép một tiến trình đang chạy trong chế độ nhân được ưu tiên
 - Không có tranh đoạt điều khiển

Giải pháp Peterson

- Giải thuật tốt cho việc giải quyết bài toán miền găng
- Áp dụng cho 2 tiến trình
- Giả sử rằng 2 lệnh **load** và **store** là ngôn ngữ máy có sẵn
- 2 tiến trình chia sẻ 2 biến
 - int turn
 - Boolean flag[2]
- Biến **turn** dùng để kiểm tra tiến trình vào CS
- Mảng **flag** được dùng xác định một tiến trình sẵn sàng vào CS chưa. **Flag[i]=true** có nghĩa là tiến trình P_i sẵn sàng

Giải thuật cho tiến trình P_i

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
        critical section  
    flag[i] = false;  
        remainder section  
} while (true);
```

Giải pháp Peterson

- Thỏa mãn 3 yêu cầu của bài toán miền găng
 - Yêu cầu về độc quyền truy xuất được đảm bảo
 *P_i vào miền găng nếu
hoặc $flag[j] = false$ hoặc $turn = i$*
 - Yêu cầu về tiến độ được thỏa mãn
 - Yêu cầu về giới hạn chờ được đáp ứng

Khóa loại trừ (Mutex locks)

- Các nhà thiết kế HĐH đã xây dựng các công cụ để giải quyết bài toán miễn găng
- Công cụ đơn giản nhất là: Khóa loại trừ (Mutex Locks – viết gọn của Mutual Exclusion)
- Bảo vệ miễn găng bằng khóa đầu tiên là ***acquire()***, sau đó là khóa ***release()***
 - Biến logic cho biết khóa có khả dụng hay không
- Giải pháp này yêu cầu cơ chế bận thì đợi (busy waiting)
 - Khóa này được gọi là spinlock

Giải pháp sử dụng khóa loại trừ

do {

acquire lock

critical section

release lock

remainder section

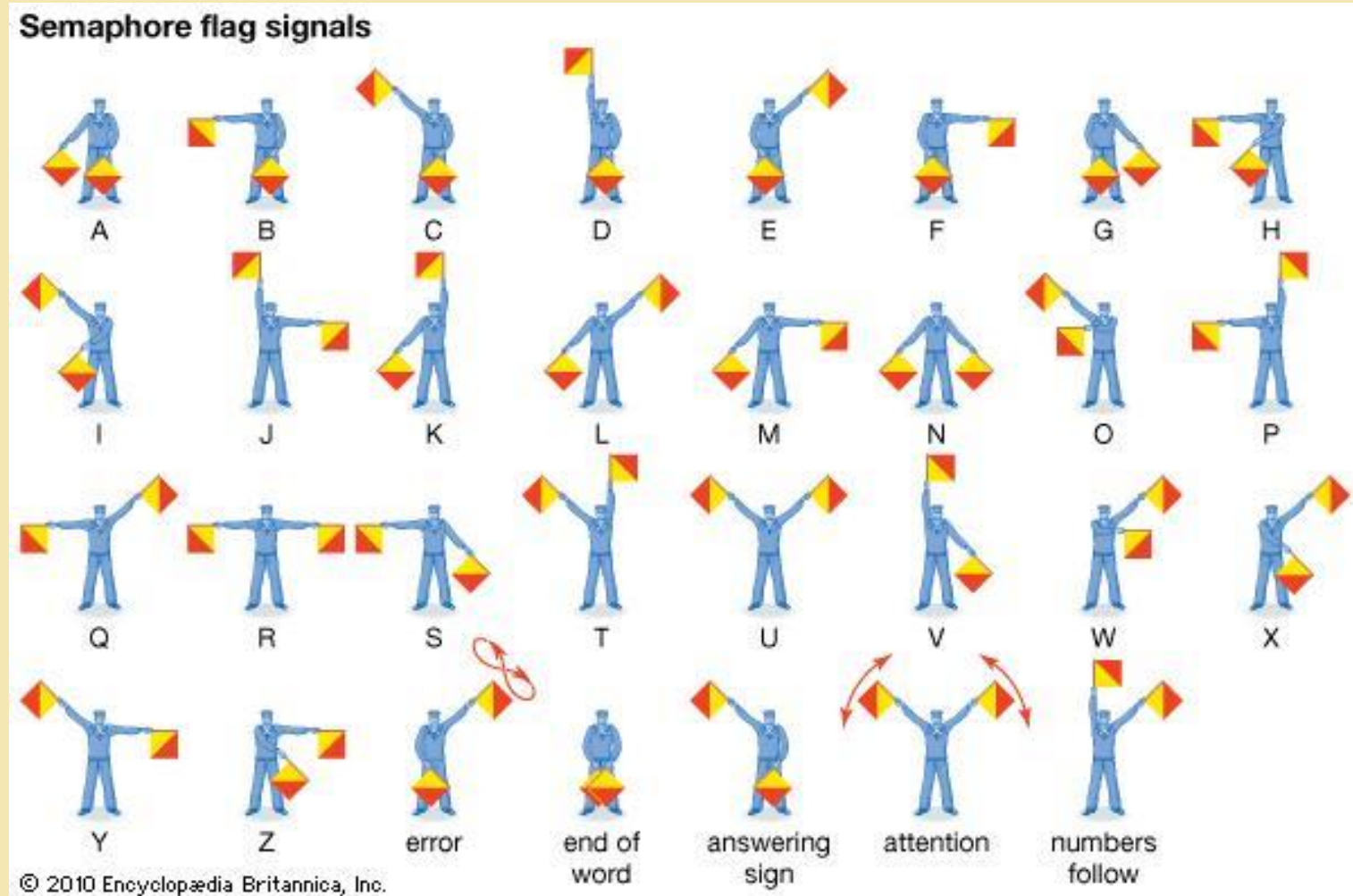
} while (true);

acquire() và release()

```
acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false;;  
}
```

```
release() {  
    available = true;  
}
```

Semaphore



Semaphore

- Edsger Wybe Dijkstra (người Hà Lan) phát minh ra khái niệm semaphore trong khoa học máy tính vào năm 1965
- Semaphore được sử dụng lần đầu tiên trong cuốn sách “The operating system” của ông



Edsger Wybe Dijkstra
(1930-2002)

semaphore

- Công cụ đồng bộ hóa cung cấp những cách tinh vi hơn cho việc đồng bộ hóa các hoạt động của tiến trình.
- Semaphore S - là một biến nguyên
- Chỉ có thể truy cập thông qua hai toán tử nguyên tử
 - **Wait()** (hoặc **P()**) và **Signal()** (hoặc **V()**)
 - P: proberen – kiểm tra (tiếng Hà Lan)
 - V: verhogen – tăng lên (tiếng Hà Lan)
- Định nghĩa toán tử wait()

```
wait(S) {  
    while (S <= 0); // busy wait  
    S--;  
}
```

- Định nghĩa toán tử signal()

```
signal(S) {  
    S++;  
}
```

Sử dụng semaphore

- Semaphore tính toán (Counting semaphore) - giá trị nguyên có thể không bị giới hạn
- Semaphore nhị phân (Binary semaphore) – giá trị nguyên chỉ là 0 hoặc 1
- Có thể giải quyết các bài toán đồng bộ hóa khác nhau

Sử dụng semaphore

- Ví dụ: xem xét 2 tiến trình chạy đồng thời: P1 với lệnh S_1 và P2 với lệnh S_2 . Giả sử chúng ta yêu cầu rằng S_2 được thực thi chỉ sau khi S_1 hoàn thành
- Tạo một semaphore "synch" khởi tạo là 0

P1:

S_1 ;

signal(synch) ;

P2:

wait(synch);

S_2 ;

Thực thi semaphore

- Phải đảm bảo rằng không có hai tiến trình có thể thực thi wait() và signal() trên cùng một semaphore vào cùng một thời điểm
- Như vậy, việc thực hiện sẽ trở thành bài toán miễn găng mà mã **wait** và mã **signal** được đặt trong CS
 - Có thể có Busy Waiting trong sự thực thi miễn găng
- Lưu ý rằng các ứng dụng có thể tốn nhiều thời gian trong CS và vì vậy đây không phải là một giải pháp tốt

Thực thi semaphore không có Busy waiting

- Với mỗi semaphore có một liên kết với hàng đợi waiting
- Mỗi phần tử trong hàng đợi waiting có hai thành phần dữ liệu:
 - Giá trị (kiểu số nguyên)
 - Con trỏ tới bản ghi kế tiếp trong danh sách
- Hai thao tác
 - Block – đặt tiến trình vào hàng đợi waiting thích hợp
 - Wakeup – lấy một tiến trình trong hàng đợi waiting và đặt nó vào hàng đợi Ready
- **`typedef struct{
 int value;
 struct process *list;
} semaphore;`**

Thực thi semaphore không có Busy waiting

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

Tắc nghẽn và đói CPU

- Tắc nghẽn (deadlock) - hai hoặc nhiều tiến trình đang chờ đợi vô hạn cho một sự kiện mà có thể được gây ra bởi chỉ một trong các tiến trình đang chờ
- Gọi S và Q là hai semaphore khởi tạo 1

P_0
`wait(S) ;`
`wait(Q) ;`
`...`
`signal(S) ;`
`signal(Q) ;`

P_1
`wait(Q) ;`
`wait(S) ;`
`...`
`signal(Q) ;`
`signal(S) ;`

- Đói CPU (Starvation) - Một tiến trình có thể không bao giờ được rời khỏi hàng đợi semaphore nơi nó bị tạm dừng

Các bài toán đồng bộ hóa kinh điển

- Bài toán vùng đệm có giới hạn
- Bài toán tiến trình đọc – ghi
- Bài toán bữa ăn tối của các triết gia

Bài toán vùng đệm có giới hạn

- N ô nhớ vùng đệm, mỗi ô chứa 1 thông tin
- Semaphore ***mutex*** được khởi tạo giá trị là 1
- Semaphore ***full*** được khởi tạo giá trị là 0
- Semaphore ***empty*** được khởi tạo giá trị là N

Bài toán vùng đệm có giới hạn

- Cấu trúc của tiến trình Producer

do {

```
    /* produce an item in next_produced */
```

```
    ...
```

```
    wait(empty);
```

```
    wait(mutex);
```

```
    ...
```

```
    /* add next produced to the buffer */
```

```
    ...
```

```
    signal(mutex);
```

```
    signal(full);
```

```
} while (true);
```

Bài toán vùng đệm có giới hạn

- Cấu trúc của tiến trình Consumer

```
do {  
    wait(full) ;  
    wait(mutex) ;  
  
    ...  
    /* remove an item from buffer to next_consumed */  
    ...  
    signal(mutex) ;  
    signal(empty) ;  
  
    ...  
    /* consume the item in next consumed */  
    ...  
} while (true) ;
```

Bài toán tiến trình Đọc - Ghi

- Một tập dữ liệu được chia sẻ giữa một số tiến trình đồng thời
 - Readers – các tiến trình chỉ đọc dữ liệu, chúng không thực hiện bất kỳ thay đổi nào
 - Writers – các tiến trình có thể đọc và ghi
- Bài toán: Cho phép nhiều tiến trình Reader đọc dữ liệu chia sẻ cùng lúc
 - Chỉ duy nhất một tiến trình Writer truy cập dữ liệu chia sẻ tại cùng thời điểm
- Bài toán tiến trình Đọc – Ghi có nhiều biến thể, tất cả đều liên quan đến một số hình thức ưu tiên
- Dữ liệu chia sẻ
 - Tập dữ liệu
 - Semaphore `rw_mutex` được khởi tạo là 1
 - Semaphore `mutex` được khởi tạo là 1
 - Integer `Read_count` được khởi tạo là 0

Bài toán tiến trình Đọc - Ghi

- Cấu trúc của một tiến trình Writer

```
do {  
    wait(rw_mutex) ;  
    ...  
    /* writing is performed */  
    ...  
    signal(rw_mutex) ;  
} while (true) ;
```

Bài toán tiến trình Đọc - Ghi

- Cấu trúc của một tiến trình Reader

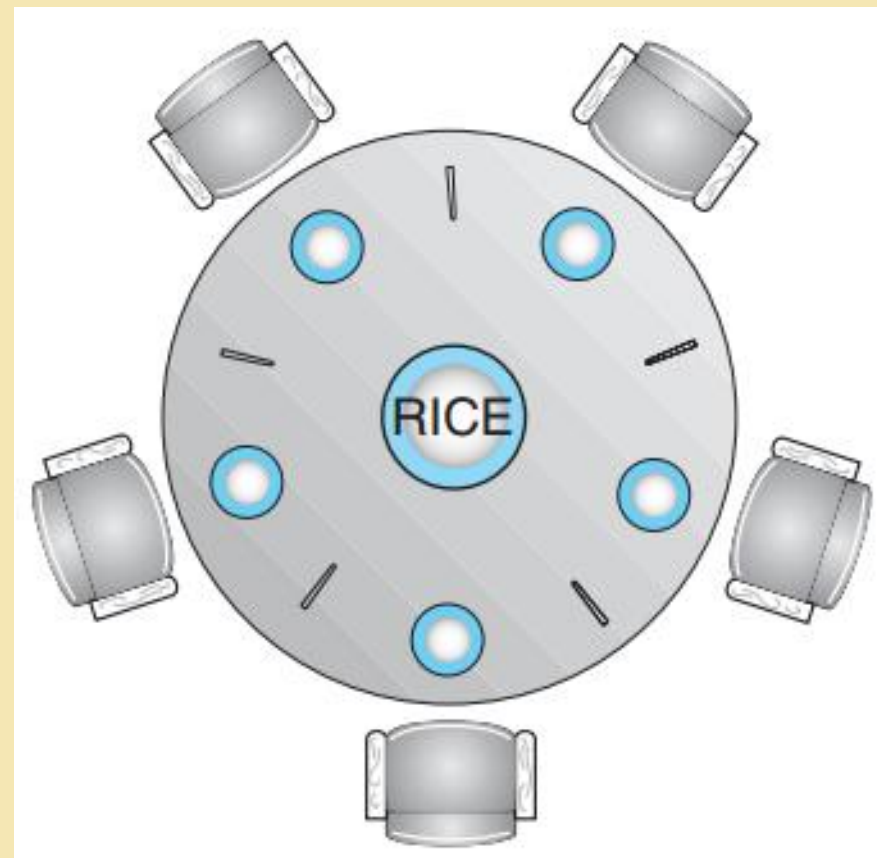
```
do {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(rw_mutex);  
    signal(mutex);  
  
    ...  
    /* reading is performed */  
    ...  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(rw_mutex);  
    signal(mutex);  
} while (true);
```


Các dạng của bài toán tiến trình Đọc - Ghi

- Dạng 1: không có tiến trình Reader phải chờ trừ khi tiến trình Writer có quyền sử dụng dữ liệu chia sẻ
- Dạng 2: khi tiến trình writer sẵn sàng, nó thực hiện việc ghi càng sớm càng tốt
- Cả 2 đều có thể xảy ra đói CPU kéo theo nhiều biến thể khác
- Bài toán tổng quát được giải quyết trên một số hệ thống bằng cách nhân HĐH cung cấp các khóa đọc - ghi

Bài toán bữa ăn tối của các triết gia

- Thuật ngữ: the dining philosophers problem
- Có 5 triết gia, 5 chiếc đũa, 5 bát cơm và một âu cơm bố trí như hình vẽ
- Đây là bài toán cổ điển và là ví dụ minh họa cho một lớp nhiều bài toán tranh đoạt điều khiển: Nhiều tiến trình khai thác nhiều tài nguyên chung



Bài toán bữa ăn tối của các triết gia

- Các triết gia chỉ làm 2 việc: Ăn và suy nghĩ
 - Suy nghĩ: Không ảnh hưởng đến các triết gia khác, đĩa, bát và âu cơm
 - Để ăn: Mỗi triết gia phải có đủ 2 chiếc đĩa gần nhất ở bên phải và bên trái mình; chỉ được lấy 1 chiếc đĩa một lần và không được phép lấy đĩa từ tay triết gia khác
 - Khi ăn xong: Triết gia bỏ cả hai chiếc đĩa xuống bàn và tiếp tục suy nghĩ

Bài toán bữa ăn tối của các triết gia

- Dữ liệu chia sẻ
 - Bát cơm (tập dữ liệu)
 - Semaphore chopstick[5] được khởi tạo là 1

Bài toán bữa ăn tối của các triết gia

- Cấu trúc giải thuật cho triết gia *i*

```
do {  
    wait (chopstick[i] );  
    wait (chopstick[(i + 1) % 5] );  
  
    // eat  
  
    signal (chopstick[i] );  
    signal (chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```

Bài toán bữa ăn tối của các triết gia

- Xử lý tắc nghẽn
 - Cho phép tối đa 4 triết gia được ngồi vào bàn cùng một thời điểm
 - Cho phép 1 triết gia lấy 2 chiếc đũa chỉ khi cả 2 đều khả thi (việc lấy đũa phải thực hiện trong miền găng)
 - Sử dụng giải pháp bất đối xứng – một triết gia có số thứ tự lẻ lấy chiếc đũa đầu tiên bên trái, sau đó chiếc bên phải. Một triết gia có số thứ tự chẵn lấy chiếc đũa đầu bên phải, sau đó chiếc bên trái.

Một số vấn đề với semaphore

- Sử dụng các toán tử semaphore không đúng
 - `signal(mutex) ... wait(mutex)`
 - `wait(mutex) ... wait(mutex)`
 - Bỏ quên toán tử `wait(mutex)` hoặc `signal(mutex)` (hoặc cả hai)
- Có thể xảy ra tắc nghẽn và đói CPU

Monitor

- Giải pháp trừu tượng hóa mức cao, cung cấp một cơ chế tiện lợi và hiệu quả cho việc đồng bộ hóa tiến trình.
- Kiểu dữ liệu trừu tượng (abstract data type - ADT), các biến nội bộ (bên trong monitor) chỉ có thể truy cập bằng mã trong thủ tục.
- Chỉ có một tiến trình duy nhất có thể hoạt động bên trong monitor tại một thời điểm.
- Nhưng không đủ mạnh để mô hình hóa một số lược đồ đồng bộ hóa (chưa có nhiều ngôn ngữ lập trình định nghĩa monitor)

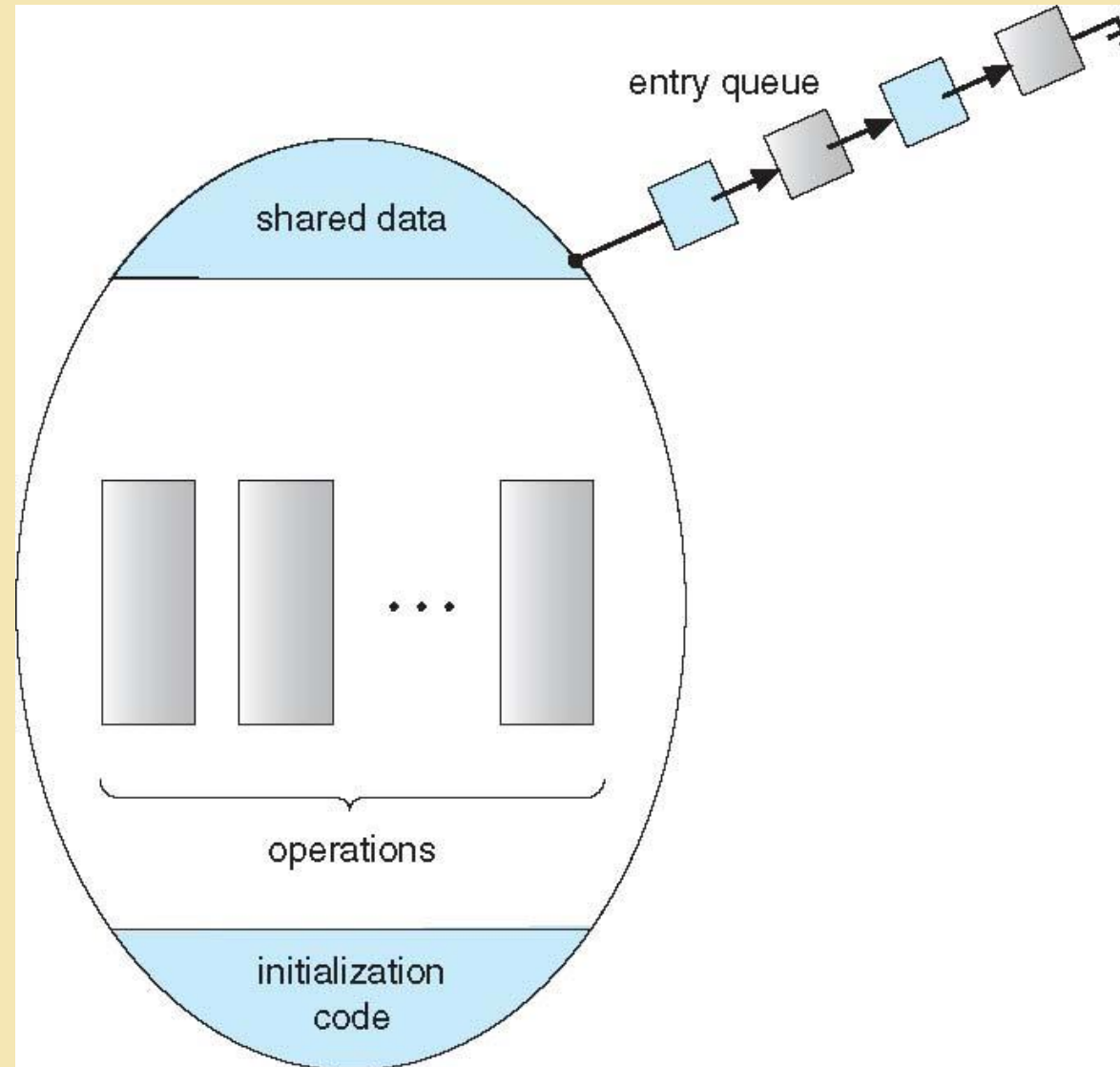
Cú pháp Monitor

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { ... }

    procedure Pn (...) {.....}

    Initialization code (...) { ... }
}
}
```

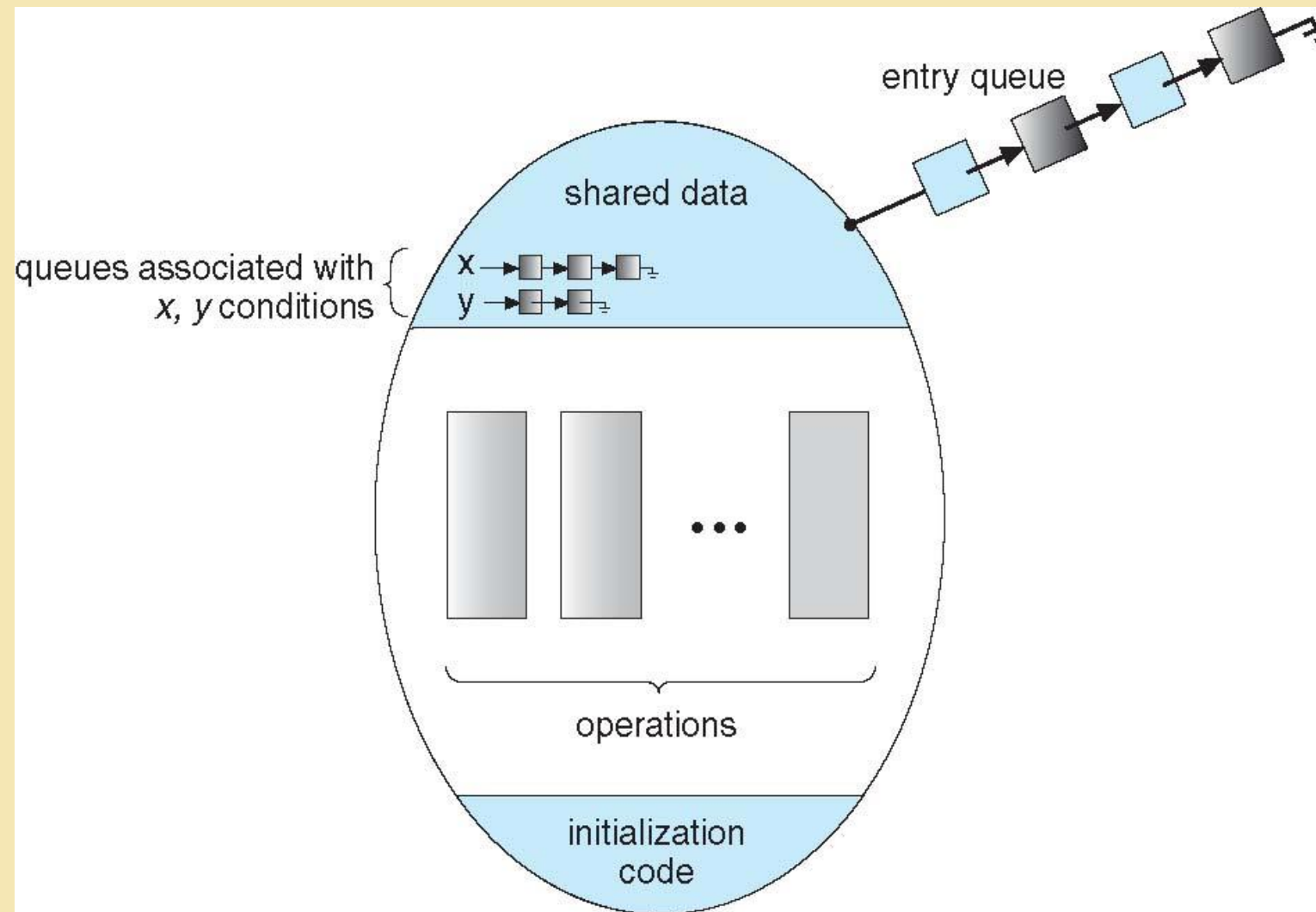
Sơ đồ của một Monitor



Biến điều kiện

- Trong một monitor, có thể định nghĩa các biến điều kiện và hai thao tác kèm theo là Wait và Signal
- **Condition x, y;**
- 2 thao tác cho phép trên biến điều kiện
 - **x.wait()** – một tiến trình gọi thao tác này tạm ngưng đến khi **x.signal()** thực thi bởi một tiến trình khác
 - **x.signal()** – khôi phục 1 trong các tiến trình đã được gọi bởi **x.wait()**
 - Nếu không có lệnh **x.wait()** thì sau đó không có tác động trên biến điều kiện

Monitor và các biến điều kiện



Lựa chọn biến điều kiện

- Nếu tiến trình P gọi `x.signal()` và tiến trình Q tạm ngưng trong `x.wait()`, thì điều gì sẽ xảy ra tiếp theo?
 - Cả 2 tiến trình Q và P không thể thực thi song song. Nếu Q được khôi phục thì P phải chờ
- Các lựa chọn:
 - Signal và wait – P chờ đến khi: hoặc Q rời monitor hoặc nó chờ một điều kiện khác
 - Signal và tiếp tục – Q chờ đến khi: hoặc P rời monitor hoặc nó chờ một điều kiện khác
 - Cả 2 đều có ưu và nhược điểm – ngôn ngữ thực thi có thể quyết định
 - Monitor được thực thi trong Concurrent Pascal theo sự thỏa hiệp
 - P đang thực thi signal rời Monitor thì Q được khôi phục
 - Được thực thi trong các ngôn ngữ khác như Mesa, C#, Java

Bài toán Dining Philosophers: giải pháp Monitor

```
monitor DiningPhilosophers
{
    enum { THINKING, HUNGRY, EATING } state [5] ;
    condition self [5];

    void pickup (int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) self[i].wait;
    }

    void putdown (int i) {
        state[i] = THINKING;
        // test left and right neighbors
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}
```


Bài toán Dining Philosophers: giải pháp Monitor

```
void test (int i) {  
    if ((state[(i + 4) % 5] != EATING) &&  
        (state[i] == HUNGRY) &&  
        (state[(i + 1) % 5] != EATING) ) {  
        state[i] = EATING ;  
        self[i].signal () ;  
    }  
}  
  
initialization_code() {  
    for (int i = 0; i < 5; i++)  
        state[i] = THINKING;  
}  
}
```

Bài toán Dining Philosophers: giải pháp Monitor

- Mỗi triết gia i gọi các hàm ***pickup()*** và ***putdown()*** theo thứ tự sau:

DiningPhilosophers.pickup(i) ;

EAT

DiningPhilosophers.putdown(i) ;

- Không có tắc nghẽn, nhưng đôi CPU có thể có

Thực thi Monitor sử dụng Semaphore

- Các biến

```
semaphore mutex;    // (initially = 1)
semaphore next;     // (initially = 0)
int next_count = 0;
```

- Mỗi thủ tục F sẽ được thay thế bởi

```
wait(mutex) ;
    ...
    body of F;
    ...
if (next_count > 0)
    signal( $\bar{next}$ )
else
    signal(mutex) ;
```

- Độc quyền truy xuất (Mutual Exclusion) trong Monitor được đảm bảo

Thực thi Monitor – biến điều kiện

- Với mỗi biến điều kiện x , ta có

```
semaphore x_sem; // (initially = 0)
int x_count = 0;
```

- Thao tác ***x.wait*** được thực thi như sau

```
x_count++;
if (next_count > 0)
    signal(next);
else
    signal(mutex);
wait(x_sem);
x_count--;
```

Thực thi Monitor – biến điều kiện

- Thao tác ***x.signal*** được thực thi như sau

```
if (x_count > 0) {  
    next_count++;  
    signal(x_sem);  
    wait(next);  
    next_count--;  
}
```

Khôi phục tiến trình trong Monitor

- Nếu một số tiến trình nằm trong hàng đợi với điều kiện x , và thực thi $x.signal$, thì tiến trình nào được khôi phục?
- FCFS không đáp ứng đầy đủ
- Xây dựng toán tử wait có điều kiện, dạng ***$x.wait(c)$***
 - c là số ưu tiên
 - Tiến trình với độ ưu tiên thấp nhất (cao nhất) được định thời kế tiếp

Cấp phát tài nguyên duy nhất

- Cấp phát tài nguyên duy nhất giữa các tiến trình cạnh tranh sử dụng số ưu tiên, quy định thời gian tối đa cho việc sử dụng tài nguyên của tiến trình

R.acquire(t) ;

**...
access the resource;
...**

R.release;

- Trong đó R là một thực thể của **ResourceAllocator**

Monitor để cấp phát tài nguyên duy nhất

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;
    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy = TRUE;
    }
    void release() {
        busy = FALSE;
        x.signal();
    }
    initialization code() {
        busy = FALSE;
    }
}
```

Đồng bộ hóa trên một số hệ điều hành

- Tham khảo "Operating System Concepts, ninth edition, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne" trang 232

Phương pháp tiếp cận thay thế

- Tham khảo "Operating System Concepts, ninth edition, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne" trang 238



Hết chương